Reporte

<table>
<tr><td colspan="3">Número del equipo: 8</td></tr>
<tr><td colspan="2">Nombres:

Esther Garza Cano

Luis Ángel David Cavazos Reyes

Emilio Garza Arizpe</td><td>Matrícula:

2143077

2142838

2143702</td></tr>
<tr><td>Nombre del curso: Componentes de Lógica de Negocio</td><td colspan="2">Nombre del profesor: Miguel Ángel Gómez Marroquín</td></tr>
<tr><td colspan="3">Actividad: Service Layer AulaPlus</td></tr>
<tr><td colspan="3">Fecha: 17/10/2026</td></tr>
<tr><td colspan="3">Bibliografia (Formato APA)</td></tr>
</table>

3. service layer

Servicio	Metodo o caso de uso	Entrada	Colaboradores	Salida	Reglas que delega
ReservaApp Service	Reservar Sesión	estudianteId, materia, tema, modalidad, fechaHora.	EstudianteRepository, TutorRepository, SesionRepository, AsignadorTutorService	SesionId	1. Valida existencia de estudiante. 2. Delega al *DomainService* buscarTutorDisponible. 3. Crea la entidad Sesion vía método de fábrica en dominio. 4. Guarda con SesionRepository.guardar(). 5. Confirma la transacción con UnitOfWork.commit().

Servicio	Metodo o caso de uso	Entrada	Colaboradores	Salida	Reglas que delega
ReservaApp Service	Solicitor cancelacion	sesionId, estudianteId, motivo, tipo	SesionRepository, NotificacionService	SolicitudId	1. Busca Sesion por ID. 2. Llama a sesion.solicitarCancelacion() donde el dominio valida el límite ≥ 48h. 3. Guarda la sesión actualizada. 4. Dispara notificación y ejecuta UnitOfWork.commit()

Servicio	Metodo o caso de uso	Entrada	Colaboradores	Salida	Reglas que delega
TutorGestionAppService	Registrar disponibilidad	tutorId, dias, horas, modalidad, zona	TutorRepository	DisponibilidadId	1. Busca Tutor por ID. 2. Llama a tutor.registrarDisponibilidad(), donde el dominio valida no solapamiento. 3. Persiste con TutorRepository.guardar(). 4. Ejecuta UnitOfWork.commit().

Servicio	Metodo o caso de uso	Entrada	Colaboradores	Salida	Reglas que delega
TutorGestionAppService	notificarAusencia	tutorId, fecha, motivo	TutorRepository, SesionRepository, NotificacionService	List<SesionId> (afectadas)	1. Llama a tutor.notificarAusencia() (dominio penaliza confiabilidad). 2. Busca sesiones afectadas del tutor. 3. Actualiza el estado de las sesiones a Cancelada/Reprogramable. 4. Aplica UnitOfWork.commit().

Servicio	Metodo o caso de uso	Entrada	Colaboradores	Salida	Reglas que delega
Evaluacion App Service	calificarSesion	sesionId, evaluadorId, puntaje, comentario	SesionRepository, TutorRepository	CalificacionId	1. Obtiene la Sesion por ID. 2. Invoca sesion.agregarCalificacion(), donde el dominio valida el máximo de 0 ... 2. 3. Actualiza el promedio en Tutor. 4. Ejecuta UnitOfWork.commit().

Servicio	Metodo o caso de uso	Entrada	Colaboradores	Salida	Reglas que delega
Liquidacion service	procesarLiquidacion	sesionId, grabacionUrl, duracion	SesionRepository, PagoGateway	LiquidacionDTO	1. Carga la Sesion. 2. Invoca sesion.completarConGrabacion() (el dominio exige ≥ 70% duración). 3. Genera el Value Object Liquidacion ($200 / $40 / $160). 4. Procesa pago en pasarela externa y hace UnitOfWork.commit().

3.1 repositorios

Repositorio del Dominio	Método / Firma del Contrato	Intención del Negocio	Parámetros y Retorno
EstudianteRepository	obtenerPorId	Recupera el agregado Estudiante registrado para gestionar sus solicitudes de reserva.	**In:** id: EstudianteId **Out:** Estudiante
EstudianteRepository	guardar	Persiste o actualiza el estado transaccional del estudiante en el sistema.	**In:** estudiante: Estudiante

			Out: void
TutorRepository	obtenerPorId	Carga el agregado Tutor para consultar o modificar su disponibilidad e índice de confiabilidad.	**In:** id: TutorId **Out:** Tutor
TutorRepository	buscarTutoresDisponibles	Ejecuta la consulta de negocio para filtrar tutores calificados según criterios del alumno.	**In:** materia, fechaHora, zona **Out:** List<Tutor>
TutorRepository	guardar	Persiste las actualizaciones del perfil, puntuación o franjas de disponibilidad del tutor.	**In:** tutor: Tutor **Out:** void
SesionRepository	obtenerPorId	Obtiene el agregado central Sesion para modificar su ciclo de vida (cancelar, calificar, liquidar).	**In:** id: SesionId **Out:** Sesion
SesionRepository	obtenerSesionesActivasDeTutor	Recupera las tutorías agendadas de un tutor para gestionar ausencias o reprogramaciones.	**In:** tutorId: TutorId, fecha: Fecha **Out:** List<Sesion>
SesionRepository	guardar	Persiste el estado actualizado de la tutoría, sus solicitudes, evaluaciones o registros financieros.	**In:** sesion: Sesion **Out:** void

2.2 Aplicación CQRS básico

Tipo de Operación	Tipo de Mensaje	Casos de Uso Representativos	Flujo Arquitectónico	Impacto en el Estado del Dominio
Escritura	**Commands** *(Comandos)*	• CrearReservaCommand • CancelarSesionCommand • RegistrarDisponibilidadCommand	Pasa por el **Application Service**, carga Agregados, valida reglas de negocio en el Dominio y persiste usando **Unit of Work**.	**Modifica Estado:** Cambia los datos en el sistema y puede emitir *Domain Events*.
Lectura	**Queries** *(Consultas)*	• ObtenerHistorialEstudianteQuery • ConsultarCatalogoTutoresQuery • VerDetalleSesionQuery	Evita el Modelo de Dominio y los Agregados complejos. Consulta directo a proyecciones o vistas rápidas para retornar **DTOs de lectura**.	**Sin Efectos Secundarios:** Operación puramente idempotente de consulta visual.

2.3 cumplimiento de los principios SOLID

Principio SOLID	Definición del Principio	Aplicación Concreta en la Arquitectura de Aula Plus	Beneficio para el Código
S - Single Responsibility *(Responsabilidad Única)*	Una clase debe tener una sola razón para cambiar.	Cada *Application Service* orquesta únicamente su módulo de casos de uso (ej. ReservaAppService). El cálculo financiero vive exclusivamente en el VO Liquidacion.	Evita clases monolíticas "Dios" y facilita el mantenimiento.
O - Open / Closed *(Abierto / Cerrado)*	Abierto a extensión, cerrado a modificación.	Si cambia una regla comercial (ej. nuevos porcentajes de comisión), se crea un nuevo *Domain Service* o *Value Object* sin modificar la capa de servicios ni repositorios.	Previene regresiones o fallas en funcionalidades existentes.
L - Liskov Substitution *(Sustitución de Liskov)*	Subtipos deben ser sustituibles por sus clases base.	Los servicios dependen de la abstracción SesionRepository. Podemos cambiar la implementación de base de datos (SQL, NoSQL, En Memoria) sin romper el servicio.	Permite intercambiar tecnologías de almacenamiento sin tocar la lógica.
I - Interface Segregation *(Segregación de Interfaces)*	Muchas interfaces específicas son mejores que una general.	Las interfaces de los repositorios son pequeñas y orientadas estrictamente a las necesidades de cada Agregado (Estudiante, Tutor, Sesion).	Evita obligar a las clases a implementar métodos que no utilizan.
D - Dependency Inversion *(Inversión de Dependencias)*	Depender de abstracciones, no de concreciones.	La Capa de Servicios y el Dominio dependen de interfaces abstractas. La infraestructura concreta (base de datos o pasarelas) implementa dichas interfaces.	Desacoplamiento total del framework y de la base de datos.